Full-Resolution U-Net with Morphological Instance Grouping for Solar Filament Segmentation

A Solution to the Solar Filament Segmentation Challenge 2026

Kushagra Tomar
Independent Researcher
Pune, India
kushagratomar2016@gmail.com

Abstract

This report describes our solution to the Solar Filament Segmentation Challenge 2026, a Kaggle competition on automatic segmentation of solar filaments in GONG H-α observations. Our pipeline has two stages. First, a U-Net with an ImageNet-pretrained EfficientNet-B3 encoder predicts a per-pixel filament probability on the full 2048×2048 image, averaged over four flipped views. Second, a short morphological post-processing chain turns this probability map into filament instances: thresholding, closing and hole filling, grouping nearby fragments through the connected components of a dilated mask, and discarding small or low-confidence components. A single model trained for 1.6 hours on one NVIDIA T4 GPU reaches a Panoptic Quality (PQ) of 0.36 on the public leaderboard and processes a test image in 1.2 seconds. The code, trained weights and a notebook that regenerates the submission without retraining are publicly available.

1 Introduction

This report goes over the details of our submitted solution to the Solar Filament Segmentation Challenge 2026 [2]. The challenge, launched on July 10, 2026, is organized by the Earth-Space AI Research (ESAIR) Lab[1]. This challenge utilizes the MAGFiLO dataset for evaluation of the segmentation algorithms [1]. Submissions are ranked by Panoptic Quality (PQ) [5], which rewards accurate masks but also penalizes every unmatched prediction and ground-truth filament. Fragmenting one filament into several pieces, or merging several filaments into one, therefore costs as much as a missed detection.

We chose a deliberately simple design: a single semantic segmentation network followed by deterministic post-processing. The network is responsible for *where* filament material is, at full image resolution so that thin structures are kept. The post-processing is responsible for *which* pixels belong to the same filament. Every step of the pipeline is contained in one Jupyter notebook[2], which loads the released weights and regenerates the submission file without retraining.

2 Methodology

Figure 1 summarizes the end-to-end pipeline. All hyperparameters are listed in Table 1.

[1] www.esairlab.com/

[2] https://github.com/KushagraTomar/Solar-Filament-Segmentation-Challenge-2026

Kaggle Competition, kaggle.com/competitions/filament-segmentation-2026 2026.

2.1 Data and Sampling

The training set contains 707 H-α images with 1154 annotator batches; an image may have been labelled independently by up to three annotators. We use each annotator batch as a separate training sample and merge all filament polygons of a batch into one binary mask. Images annotated by several experts are therefore seen several times per epoch with slightly different labels, which acts as a mild form of label smoothing at filament boundaries. Only the provided H-α images and segmentation polygons are used; no external data and no other annotation fields (bounding boxes, spines, labels) are used.

The final model is trained on all 1154 samples with no held-out split, so that every annotated image contributes to the released weights. The post-processing parameters (Section 2.6) were selected by a grid search on validation PQ using a held-out split in earlier development experiments, and were then kept fixed.

2.2 Pre-processing

Images are read as 8-bit grayscale and normalized with the grayscale ImageNet statistics ($\mu = 0.449, \sigma = 0.226$). We do not resize, crop the solar disk, or correct limb darkening: the network sees the original pixels, which keeps narrow filaments and barbs that would be lost by downsampling a 2048×2048 image.

2.3 Segmentation Model

We use a U-Net [9] with an EfficientNet-B3 encoder [11] pretrained on ImageNet, as implemented in `segmentation_models_pytorch` [4]. The first convolution is adapted to a single input channel, and the decoder ends in a single output channel holding the filament logit. The model has about 13 M parameters (53 MB in 32-bit floats). Because it is fully convolutional, it is trained on crops but applied to the whole image at inference.

2.4 Training

Each sample is augmented with a random 1024×1024 crop, horizontal and vertical flips, 90° rotations, a mild affine transform (scale 0.9–1.1, rotation ±15°), brightness/contrast and gamma jitter, and occasional Gaussian blur, using Albumentations [3]. The loss is the sum of binary cross-entropy and soft Dice loss [8]; the Dice term counteracts the strong foreground–background imbalance, since filaments cover a small fraction of the disk. We optimize with AdamW [7] and a one-cycle learning-rate schedule [10] in mixed precision for 30 epochs (288 iterations of batch size 4 per epoch). We keep the weights from the final epoch, where the schedule ends at a near-zero learning rate. The training loss falls from 1.85 in the

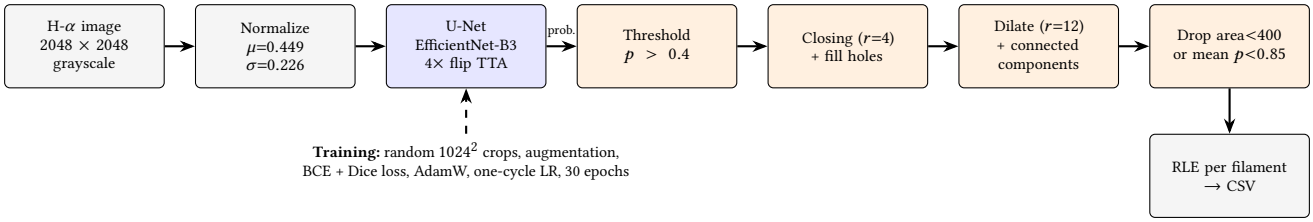


Figure 1: End-to-end pipeline. Blue: learned model. Orange: deterministic post-processing that converts the probability map into filament instances. All radii and areas are in pixels of the original 2048×2048 image.

Table 1: Hyperparameters of the final pipeline.

Setting	Value
Encoder / weights	EfficientNet-B3 / ImageNet
Input	1 channel, full resolution
Training crop / batch size	1024^2 / 4
Optimizer / LR / weight decay	AdamW / 3×10^{-4} / 10^{-4}
Schedule / epochs	one-cycle / 30
Loss	BCE + Dice
Test-time augmentation	4 flips (none, H, V, H+V)
Threshold τ	0.4
Closing radius r_c	4 px
Merge (dilation) radius r_m	12 px
Minimum area $A_{\min}$	400 px
Minimum mean probability $c_{\min}$	0.85
Random seed	42

first epoch to 0.31 in the last. Training takes 1 h 37 min on a single NVIDIA T4 (16 GB).

2.5 Inference

The whole 2048×2048 image is passed through the network four times (original, horizontal flip, vertical flip, both), each output is flipped back, and the four sigmoid probability maps are averaged into a single map P.

2.6 Post-processing: from Pixels to Filaments

PQ is computed on instances, so the probability map must be split into individual filaments. With D_r a disk of radius r, we compute:

(1) **Threshold**: $M = [P > \tau]$.
(2) **Smooth**: $M \leftarrow \text{fill}(M \bullet D_{r_c})$, a morphological closing followed by hole filling. The MAGFiLO masks are rasterized polygons with no holes, so small gaps and holes in M can only be errors.
(3) **Group**: label the 8-connected components of the dilated mask $M \oplus D_{r_m}$, then keep the labels only on the pixels of M. Fragments separated by less than about $2r_m$ pixels share one identifier, which reduces one-to-many matches, while each instance keeps its undilated shape. An instance may therefore consist of several disconnected parts.
(4) **Filter**: discard an instance if its area is below $A_{\min}$ or its mean probability is below $c_{\min}$. In PQ, every false-positive instance increases the denominator, so we prefer to drop small or uncertain blobs.

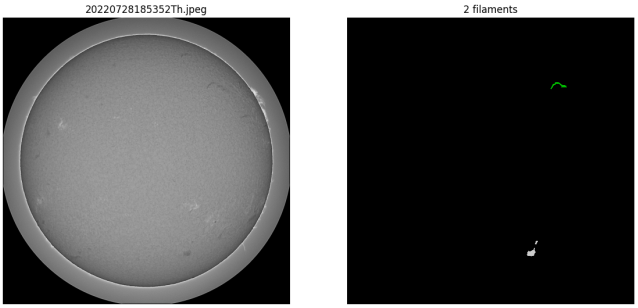


Figure 2: Test image 20220728185352Th (left) and the predicted instances (right; one color per filament). The lower instance shows the grouping step: two nearby fragments are assigned the same identifier.

Table 2: Results of the submitted pipeline (single NVIDIA T4 GPU).

Metric	Value
Public leaderboard PQ	0.36
Training time	1 h 37 min
Inference, 180 test images	222 s (1.23 s / image)
Model size	≈13 M parameters, 53 MB

Each remaining instance is encoded as COCO run-length counts with `pycocotools` [6] and written as one row of the submission file. The few test images with no detection receive a single empty mask, so that every image appears in the file. Figure 2 shows the result on a test image.

3 Evaluation

3.1 Strategy

The final model was evaluated on the hidden test set through the competition leaderboard, which reports PQ on about half of the 180 test images. Test ground truth is not available to participants, and the released model uses all annotated images for training. We therefore report the leaderboard score, the computational cost, a reproducibility check, and descriptive statistics of the predictions (Tables 2 and 3). We do not report per-image Dice or IoU distributions or error bars on PQ, because they would require ground truth that the final model has not seen.

Table 3: Statistics of the predictions on the 180 test images.

Quantity	Value
Predicted filaments	1037
Filaments per image, mean ± SD	5.76 ± 3.36
Filaments per image, median (IQR)	6 (3–8), max 14
Images with no detection	4
Filament area, median (IQR)	1542 (851–2792) px
Filament area, range	400–20788 px

3.2 Computational Efficiency

The inference time in Table 2 covers the whole per-image pipeline: reading the image, four forward passes, post-processing and RLE encoding. A single network with no ensembling, a fixed-cost post-processing chain and no resizing keep the method simple and fast.

3.3 Reproducibility

The submitted file was generated by the released notebook from the released weights, without retraining. We compared it with the predictions written at the end of the training run, made with the same weights in memory. Both contain the same 1041 rows. Of these masks, 968 are bit-identical; the remaining 73 differ only by a few boundary pixels (104 of 2.38 M predicted pixels, IoU ≥ 0.994 for every mask). We attribute these differences to non-deterministic mixed-precision GPU arithmetic; they do not change the number of filaments.

3.4 Strengths and Weaknesses

Strengths. (i) Full-resolution training and inference keep thin filaments that downsampling would erase. (ii) The pipeline is small and fast: one network, flip TTA and five post-processing steps with interpretable parameters. (iii) Training on every annotator batch uses all expert labels without having to resolve disagreements between them. (iv) Results are reproducible from the released weights.

Weaknesses. (i) The network is trained for semantic segmentation only; filament identity is decided by a fixed merge radius. This can join two distinct filaments that lie close together (many-to-one), or fail to join a filament with gaps larger than $2r_m$ (one-to-many). (ii) The minimum area of 400 px removes small filaments together with noise. (iii) One global threshold is used for images from six GONG sites with different seeing conditions and contrast. (iv) Without an internal validation of the final model, we cannot attach uncertainty to the leaderboard score.

Possible improvements. K-fold cross-validation would provide per-image PQ, Dice and IoU distributions with bootstrap confidence intervals, and a fold ensemble would likely improve accuracy. Predicting instance cues directly, for example filament spines or boundaries, would let grouping depend on filament orientation and continuity instead of distance alone. Per-site normalization or threshold calibration could reduce the effect of varying image quality.

4 Acknowledgment

The organization of the Filament Segmentation Challenge 2026 is partially supported by the U.S. National Science Foundation (NSF) under Grant No. 2209912 and 2433781, directorate for Computer and Information Science and Engineering (CSE), and Office of Advanced Cyberinfrastructure (OAC), and Grant No. 2511630, AST Division Of Astronomical Sciences and MPS Directorate for Mathematical and Physical Sciences.

The Filament Segmentation Challenge 2026 is sponsored by the U.S. National Science Foundation (NSF) National Solar Observatory (NSO).

This work utilizes GONG data obtained by the NSO Integrated Synoptic Program, managed by the National Solar Observatory, which is operated by the Association of Universities for Research in Astronomy (AURA), Inc. under a cooperative agreement with the National Science Foundation and with contribution from the National Oceanic and Atmospheric Administration. The GONG network of instruments is hosted by the Big Bear Solar Observatory, High Altitude Observatory, Learmonth Solar Observatory, Udaipur Solar Observatory, Instituto de Astrofísica de Canarias, and Cerro Tololo Interamerican Observatory.

References

[1] Azim Ahmadzadeh, Rohan Adhyapak, Kartik Chaurasiya, Laxmi Alekhya Nagubandi, V. Aparna, Petrus C. Martens, Alexei Pevtsov, Luca Bertello, Alexander Pevtsov, Naomi Douglas, Samuel McDonald, Apaar Bawa, Eugene Kang, Riley Wu, Dustin J. Kempton, Aya Abdelkarem, Patrick M. Copeland, and Sri Harsha Seelamneni. 2024. A dataset of manually annotated filaments from H-alpha observations. In *Nature Scientific Data (Nature Scientific Data, Vol. 11)*. Springer, Article 1031, 1031 pages. doi:10.1038/s41597-024-03876-y

[2] Azim Ahmadzadeh, Dustin J. Kempton, Qin Li, and Alexei A. Pevtsov. 2026. Solar Filament Segmentation Challenge 2026. https://kaggle.com/competitions/filament-segmentation-2026. Kaggle.

[3] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. 2020. Albumentations: Fast and Flexible Image Augmentations. *Information* 11, 2 (2020), 125. doi:10.3390/info11020125

[4] Pavel Iakubovskii. 2019. Segmentation Models Pytorch. https://github.com/qubvel-org/segmentation_models.pytorch.

[5] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother, and Piotr Dollár. 2019. Panoptic Segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 9404–9413. doi:10.1109/CVPR.2019.00963

[6] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. 2014. Microsoft COCO: Common Objects in Context. In *European Conference on Computer Vision (ECCV) (LNCS, Vol. 8693)*. Springer, 740–755.

[7] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations (ICLR)*.

[8] Fausto Milletari, Nassir Navab, and Seyed-Ahmad Ahmadi. 2016. V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation. In *Fourth International Conference on 3D Vision (3DV)*. 565–571. doi:10.1109/3DV.2016.79

[9] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI) (LNCS, Vol. 9351)*. Springer, 234–241.

[10] Leslie N. Smith and Nicholay Topin. 2019. Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications (Proc. SPIE, Vol. 11006)*. 1100612. doi:10.1117/12.2520589

[11] Mingxing Tan and Quoc V. Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 6105–6114.